5,6주차 에세이

김희창

1. Gemini space invader 베릴로그 코드에서 놓친 부분들

always 블록의 안에서 연속 할당(blocking assignment)가 있는 부분이 문제가 있다. 또한 하나의 always 블록 안에 많은 코드를 넣어 놓았는데, 이는 소프트웨어에 가까운 구현 방식으로, 하드웨어 구현에서는 독립적인 기능별로 다른 always 블록을 작성하는 편이 더 좋다고 한다.
이처럼 LLM은 소프트웨어 코드는 잘 작성하지만, 하드웨어 코드는 소프트웨어 처럼 작성하는 경향이 있기 때문에 주의를 기울여서 LLM이 작성한 베릴로그 코드를 살펴보아야한다. 이를 위해서는 베릴로그 코드에 대한 이해가 필요하다.

2. 시뮬레이션, 에뮬레이션, 프로토타이핑의 비교

이번 주 가장 큰 궁금증은 에뮬레이션과 프로토타이핑이 어떻게 다른지였다. 이에 대해서 이해한 부분을 나의 언어로 정리해보았다.

첫째, 시뮬레이션은 소프트웨어에서 모든 것을 하는 것으로, 베릴로그 언어로 작성한 것을 회로로 합성하지 않고 마치 소프트웨어인 것처럼 실행하는 것이다. 회로를 설계할때 거치는 첫 단계로, 사용자가 만들고자 하는 로직을 구현한 후 의도한 대로 동작하는지 확인한다. 시뮬레이션에서는 시간이 설정한 일정 간격으로 이산적으로 증가하면서 모든 신호가 베릴로그의 구문에 의해 정해진 규칙으로 변화한다. 또한 각 시각에서 플립플롭에 저장된 값을 기록하고 분석할 수 있다. 따라서 이 과정에서 디버깅을 통해서 오류를 쉽게 수정할 수 있다.

둘째 단계는 (코)에뮬레이션으로, 설계한 베릴로그 코드를 FPGA에 합성을 해서 올리고, 입력과 출력은 소프트웨어를 통해서 제어하는 방식이다. 베릴로그의 구문은 합성기에 의해서 실제 회로로 합성이 될 수 있는데, FPGA를 타겟으로 하면 플립플롭과 LUT을 이용한 회로가 만들어진다. 이 회로의 정보를 FPGA에 업로드하여 설정하고, 입력 신호를 넣어주어 원하는 출력 신호가 나오는지 확인한다. 입출력 신호를 위해서는 소프트웨어를 사용하는데, PC의 신호를 FPGA의 핀에 입력하기 위해서는 중간에 인터페이스의 역할을 하는 컨트롤러가 필요하다.
우리 수업에서는 라즈베리파이 Pico를 통해 PC와 USB Serial 통신의 결과를 디지털 신호 핀으로 전달해주는 MI(Modeling Interface)를 사용하였다. PC에서 클록과 함께 비롯한 입력 신호를 전달하면, 이를 Pico가 이를 3.3V 디지털 신호로 바꾸어서 FPGA에 입력해주고, 다시 FPGA의 출력 핀을 읽어서 출력 신호를 PC에 전달해준다. PC는 이 값을 확인 및 저장하고, 다음 입력 신호를 전달한다. 이 과정을 거쳐서 에뮬레이션이 이루어진다. 따라서 입력과 출력 값은 모든 클록에서 확인이 가능하다. 그러나 회로의 내부 레지스터 값은 접근이 불가능하거나 어렵다는 한계가 있다.
에뮬레이션을 하는 이유는 시뮬레이션 보다 더 실제 회로에 가까운 상황에서도 기능이 정상적으로 작동하는지 확인하기 위함이다. 에뮬레이션에서는 실제 전자회로를 사용하기 때문에 시간이 연속적으로 증가하고, 이에 따라서 race condition이 발생하거나 신호의 전달이 지연되어서 잘못된 값이 입력되는 등의 문제가 발생할 수 있다. 이는 실제 칩에서도 발생할 수 있는 문제이므로 에뮬레이션을 통해 사전에 알아차릴 필요가 있다.

셋째 단계는 프로토타이핑으로, 에뮬레이션의 FPGA를 사용하고 입력과 출력 신호까지 모두 마이크로컨트롤러에서 주는 방식이다. 실제 칩을 구동하는 환경에서는 PC가 없기 때문에, 칩의 구동기 역할을 할 수 있는 마이크로컨트롤러가 필요하다. 프로토타이핑을 하면 이를 사용하여 실제와 같은 환경에서 테스트를 해본다. 클록을 비롯한 입력 신호를 마이크로컨트롤러에서 발생시키기 때문에 PC와 Serial 통신을 할 필요가 없어서 더욱 빠른 속도의 동작이 가능하다. 그러나 PC에서 입력 및 출력 값을 분석 및 저장할 수 없다. 따라서 신호의 파형을 알기 위해서는 오실로스코프를 사용해야 한다. 프로토타이핑은 칩 제작을 의뢰하기 전 마지막 단계로, FPGA를 칩으로 대체하였을때 마이크로컨트롤러를 비롯한 전체 구성이 올바르게 작동함을 확신할 수 있어야 한다.
우리 수업의 프로토타이핑에서는 키보드 입력을 받기 위해서 여전히 PC에 연결을 하였으나, PC는 주변장치의 역할을 할 뿐이고 중심 구동은 Pico에서 이루어진다.

시뮬레이션에서 에뮬레이션, 그리고 프로토타이핑으로 진행할수록 점점 실제 회로에 가까워지지만, 디버깅을 하고 설계를 고치기 위해서 점점 더 많은 노력을 필요로하게 된다. 따라서 각 단계에서 버그가 없이 동작함을 확인하고서 넘어가는 것이 중요하다.

3. 우주선이 우주괴물 파괴 시 효과음이 발생하도록 만들기

주어진 코드와 PSCE-MI 를 통한 standard co-emulation, 프로토타이핑을 더 잘 이해하기 위해서 한개의 출력 신호를 추가해보는 변형을 해보았다. 가장 간단하게 우주선의 총알이 우주괴물을 파괴할 시 1 이 출력되는 hit 출력을 만들고, 이것이 PC에 전달되어 효과음('\a' 출력 시의 벨소리)이 재생되도록 하는 것을 1차 목표로 삼았다.

3-1. 시뮬레이션
우선 베릴로그 모듈에 출력 hit 신호를 간단히 추가해주었다.
이를 위해서 output reg hit;을 정의하고, always@ 블록 내에 충돌 여부를 기록하는 코드 두 줄을 추가하였다.

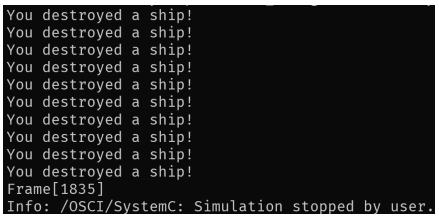
```
hit <= 1'b0;
if (bullet_active && invader_alive[chk_idx]) begin
    if ((bullet_x >= calc_inv_x) && (bullet_x <= calc_inv_x + OBJ_WIDTH) &&
        (bullet_y >= calc_inv_y) && (bullet_y <= calc_inv_y + OBJ_HEIGHT)) begin
            invader_alive[chk_idx] <= 1'b0;
            bullet_active          <= 1'b0;
            hit                    <= 1'b1;
    end
end
```

또한, simulation 디렉토리의 systemC testbench에도 hit 신호가 1이 될 경우 “You destroyed a ship!” 과 알림음을 콘솔에 출력하는 스레드를 추가하였다.

```
void sc_glcd128x64_TLM::Hit_Thread(void)
{
        while(true){
                wait(clk_o.posedge_event());
                if(hit.read()){
                        printf("You destroyed a ship!\a\n");
                }
```

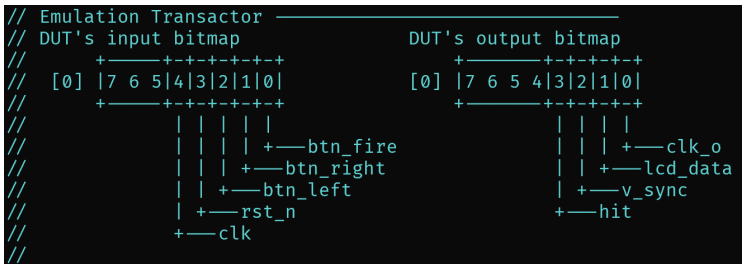
```
        }
}
```

시뮬레이션 결과 다음과 같이 원하는 대로 hit 신호가 1이 되는 사이클에 메시지와 벨소리가 출력되었다.



3-2. 에뮬레이션

우선 space_invaders_engine_glcd_wrapper.v 에서 아래 도표와 같이 output의 3번 비트에 신호 hit를 추가해주었다.



이후 PSCE-TRANS 디렉토리를 들어가서 변경해야 할 부분을 살펴보았는데, 놀랍게도 전혀 변경 부분이 없었다. tcl파일에 주어진 핀 할당 설정은 “set_location_assignment PIN_132 -to Dout_emu[7]”와 같이 standard emulation interface를 기준으로 작성되어 있어서 개별 핀을 추가한다고 변경하지 않아도 되었다. 내부의 베릴로그 모듈만 교체하고 다시 합성을 해주면 더 이상 변경할 부분이 없었다. 이것이 SCE-MI를 사용할 때의 또 하나의 장점인 것 같다. 마찬가지로 PSCE-MI 디렉토리에서도 변경할 부분이 없었다.

Espace_invaders_engine_glcd.h 에는 hit 핀을 추가해주는 마찬가지의 수정을 해주었다. 이 SystemC 파일에서는 wrapper 기능 뿐 아니라 Pico의 시리얼 통신과 관련된 파일 입출력 기능도 추가되어 있는 것을 관찰하였다. 그러나 space_invaders_engine_glcd 모듈과 직접 연결되어 있지 않았는데, 이는 에뮬레이션에서는 모듈이 FPGA상에 존재하기 때문에 시리얼 통신으로 연결되어있기 때문이다. SystemC 테스트벤치 관점에서는 이 모듈이 sc_glcd128x64_TLM 과 연결되어있고, 시리얼 통신은 외부 입출력으로 보일 것이다. 에뮬레이터의 Makefile에는 이 헤더파일을 추가하여 컴파일 하도록 설정되어있었다.

업로드하여 확인한 결과 실행 속도가 느리지만 마찬가지로 “You destroyed a ship!” 이 나오는 것을 확인하였다.

3-3. 프로토타이핑

프로토타이핑에서는 SCE-MI의 규약을 더 이상 사용하지 않기 때문에 Pico와 FPGA의 핀 번호를 지정해 주어야한다. PIN_HIT를 Pico의 GPIO_13번 핀으로 설정하고, 회로도를 따라가서 PCB의 J2 커넥터의 6번 핀으로, FPGA의 PIN_124 으로 지정해주었다.

우선 Pico의 펌웨어 코드서 attachInterrupt(digitalPinToInterrupt(PIN_HIT), handlerHit, RISING); 을 이용해서 PIN_HIT의 값이 1이 되는 경우 실행되는 핸들러에서 시리얼 출력 'H'을 내보내도록 변경하였다.
FPGA는 Makefile을 통해서 wrapper를 제거한 모듈을 합성하였다. 이때 .tcl 파일에서 pin assignment를 "set_location_assignment PIN_124 -to hit" 와 같이 추가해주었다.

systemC 코드의 경우 PC입장에서 시리얼 입력 신호가 생길 수 있기 때문에, 루프를 돌면서 입력 신호가 있는 경우 이를 읽어서 'H'가 맞는지 확인하고, hit을 HIGH로 설정하는 아래 코드를 추가하였다.

```
        // Receive, if unread byte exists
        int available = 0;
        if(ioctl(fd, FIONREAD, &available) == -1){
                perror("FIONREAD");
        } else if (available > 0 && read(fd, &_Rx_, 1) == 1){
                hit.write(_Rx_=='H' ? true : false);
        } else {hit.write(false);}
```

이후 hit 신호를 테스트벤치에 추가해주고, 다음과 같은 u_sc_effect 모듈을 추가해서 clk, hit을 입력으로 받아서 hit이 HIGH인 경우 콘솔에 출력을 내보내도록 하였다.

```
// sc_effect.h
SC_MODULE(sc_effect)
{
    sc_in<bool>             clk;
    sc_in<bool>             hit;


    void Hit_Thread(void){
        while(true){
                if(hit.read()){
                        printf("You destroyed a ship!\a\n");
                }
                wait(100, SC_NS);
        }
    }


    SC_CTOR(sc_effect)
    {
        SC_THREAD(Hit_Thread);
        sensitive << clk;

    }
};
```

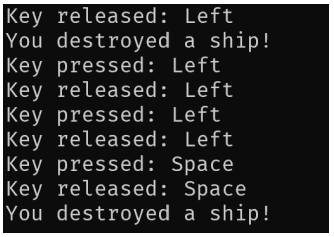
작동시켜본 결과, 키보드 입력을 받아 플레이어가 움직였으나, 우주선을 파괴하여도 메시지가 나오지 않았다?

우선 _Rx_를 출력하는 방식으로 디버깅해본 결과, 시리얼 통신이 PWM과 관련된 신호를 보내는 것이 포착되었다. 따라서 space_invaders_thread의 시작 지점을 시리얼 통신이 형성된 이후로 변경해주었다. 그러나 여전히 같은 문제가 반복되었다. 운이 좋게도 PWM신호에 'H'가 포함되지는 않아서 hit이 예상치 못하게 HIGH가 되는 일은 없었다. 조금 더 플레이해보니 어떤 경우는 우주괴물을 파괴했을때 hit이 나오고 어떤 경우는 인식하지 않는 문제가 있었고, 이것이 hit의 rising edge가 너무 짧아서 Pico가 인식하지 못하기 때문이라 추측하였다. 화면이 좌우로 흔들리는 것 역시 vsync의 rising edge를 Pico가 잘 인식하지 못하고 있는 것이라 생각하였다. 이 가설을 검증하기 위해서 clock의 주파수를 100kHz에서 10kHz으로 하여서 주기를 10배, 즉 rising edge의 길이를 10배로 증가시켜 실험해보았다. 아쉽게도 화면이 좌우로 흔들리는 문제는 사라졌으나 Hit이 인식되지 않는 것은 같은 문제가 반복되었다..

결론적으로 인식이 되지 않는 원인은 interrupt 내부에서 Serial.write를 호출하는 코드가 너무 오랜 시간이 걸리기 때문이었다. serial.write 대신 hit_detected=true 플래그를 설정하고, 메인 루프에서 Serial.write를 보내주니 대부분의 경우 인식이 잘 되었다.

```
void loop(void)
{
  while(true)
  {
    if (Serial.available())
    {
      rxData = Serial.read();
//...//
    }
    if(hit_detected){
        Serial.write('H');
        hit_detected=false;
    }
}
void handlerHit()
{
  hit_detected=true;
}
```

다음 사진과 같이 SDL 키보드 입력과 우주선을 파괴했을때 나오는 효과음이 프로토타입에서도 잘 동작하는 것을 확인하여 뿌듯하였다.



3-4. 후기

기능을 추가하기 위해서 출력 핀을 1개 늘려보니 에뮬레이션과 프로토타이핑에 대해서 더 잘 이해할 수 있었다. 특히 에뮬레이션에서는 PSCE-MI라는 규약을 사용해 베릴로그의 wrapper와 systemC의 코드를 이에 대응시킴으로써, 물리적인 핀의 연결에 대해서 고민할 필요가 없었다. 따라서 하나의 Pico 펌웨어 코드를 가지고 여러가지 다른 베릴로그 회로를 테스트할 수 있다.
프로토타이핑에서는 직접 FPGA의 핀 번호와 제어할 Pico의 핀 번호를 지정해 주었고, 시리얼 통신 으로 데이터를 전달하는 형식 역시 사용자가 원하는 대로 바꿀 수 있었다. 나의 경우는 에뮬레이션까지 문제 없이 잘 진행되다가 프로토타이핑에서 문제가 발생하였는데, 이 경우 어디가 문제인지 디버깅하기가 까다로웠다. 그래도 에뮬레이션을 통해서 FPGA 자체의 동작에는 문제가 없음을 확인했으므로, Pico의 펌웨어와 SystemC 코드만 디버깅을 해보면 되었다. 만약 이와 같은 상황에서 에뮬레이션을 하지 않고 바로 프로토타이핑을 진행하였다면, 문제가 될 수 있는 지점이 FPGA, Pico, SystemC, 시리얼 통신 등이 전부 가능하여서 오류를 찾기가 정말 힘들었을 것이다. 따라서 시간이 많이 들더라도 에뮬레이션을 하는 것이 오히려 최종적으로 오류를 줄일 수 있는 더 빠른 길이라는 생각이 들었다.

4. 실습과 과제

Q14 디렉토리에서 나누어준 FPGA 보드에 에뮬레이션과 프로토타이핑을 실습하였다.

에뮬레이션에서는 PSCE-MI와 PSCE-TRANS 폴더를 통해 각각 Pico와 FPGA(Altera Cyclone IV) 를 프로그래밍해주었다. 이후 SystemC 테스트벤치를 빌드하고 실행하였다.

프로그래밍 방법은 Pico의 경우 보드의 Bootsel 버튼을 누른 채로 reset 버튼을 눌러서 드라이브로 인식되도록 한 이후에, sudo mount -t drvfs f: /mnt/f 를 통해서 f 드라이브로 게스트 OS인 WSL에 Pico를 마운트한 이후 바이너리를 업로드해주었다. FPGA의 경우 cmd에서 usbipd attach를 통해 게스트 OS에 연결하고 openFPGALoader를 이용해서 *.rbf 파일을 업로드해주었다. 학습노트의 설명을 따라하니 문제없이 업로드할 수 있었다. 이후 Pico의 시리얼 통신을 활성화하기 위해 한번 더 usbipd attach를 해주었다.

에뮬레이션에서는 lcd 화면과 PC에 모두 게임 화면이 출력되었으나, 한 프레임이 지나가는데 수 초가 걸릴 정도로 느리게 실행이 되었다. 이는 USB Serial 통신의 한계 때문으로, USB의 polling rate는 기본 125Hz, 최대 수 kHz로 매우 낮기 때문에 한 클록에 1ms가 걸린다고 해도 1프레임인 8192클록을 진행하기 위해서는 8초나 소요가 된다. USB로 평소에 파일을 전송할때는 제어신호 한번에 데이터를 많이 보내기 때문에 수 Gbps의 전송률이 가능하지만, 에뮬레이터로 사용할때는 매 클록마다 신호를 주고 받아야 하므로 제어신호의 속도에 의해 통신이 제한된다.

프로토타이핑은 application 디렉토리에서 진행하였다. firmware, fpga, run 폴더를 통해 각각 Pico, FPGA를 프로그래밍하고, Pico에 키보드 인터페이스를 연결하였다. 프로그래밍 방법은 에뮬레이션과 같다.

실행 결과 PC 화면에는 아무것도 표시되지 않고, 보드의 lcd에 게임 화면이 표시가 되었다. 초당 여러 프레임이 업데이트가 되어서 정상적인 게임 진행이 가능하였고, 화살표 키를 누르면 우주선이 좌우로 움직이는 것을 확인하였다.

그러나 게임 화면이 좌우로 흔들려서 출력되는 문제가 있었다. 시작 화면에서도 신호가 타이밍이 잘 안맞는지, 격자 무늬에서 몇몇 픽셀이 잘못된 색을 출력하였는데, 원인을 알아보아야한다. Serial 통신을 제거하면 화면이 정상 출력되는 것으로 보아 Pico에 문제가 있는 것으로 추정된다.

